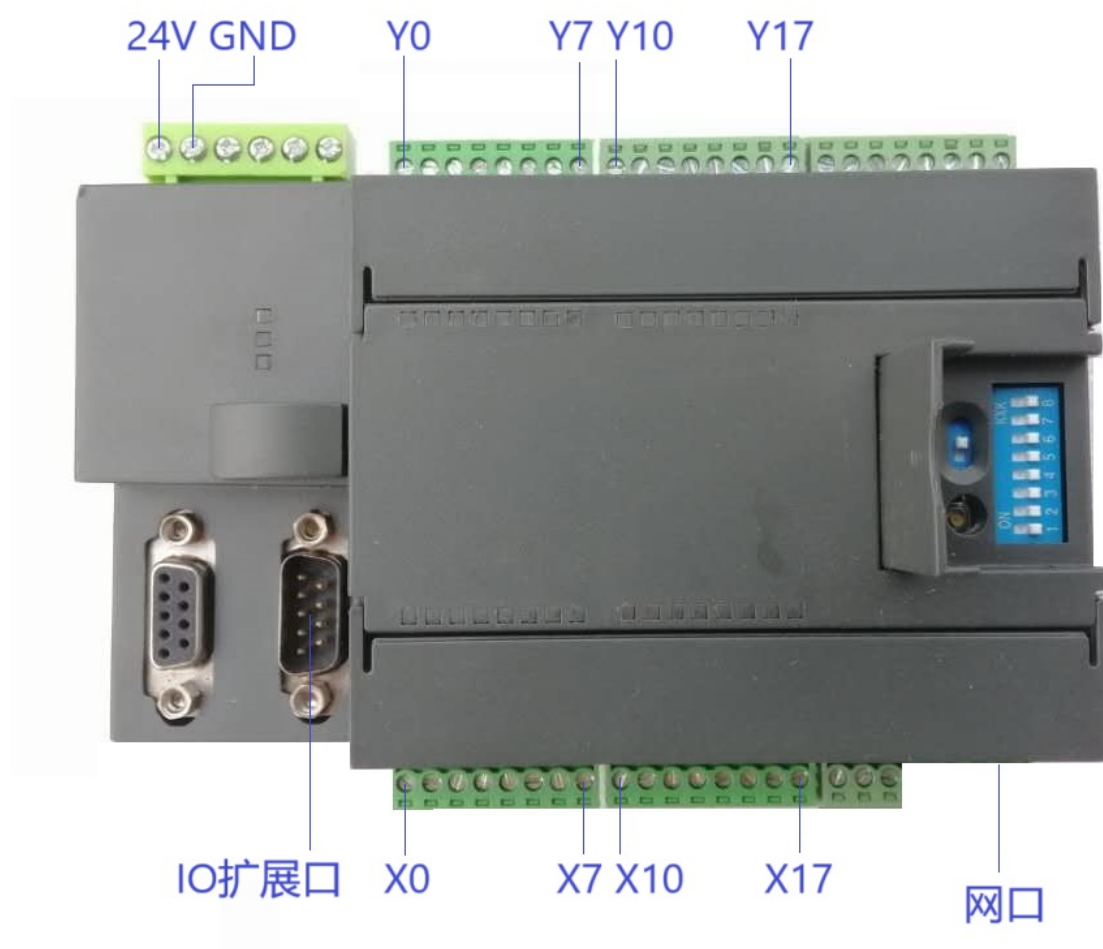


Eth-IO-AD-DA 用户手册 V2.0

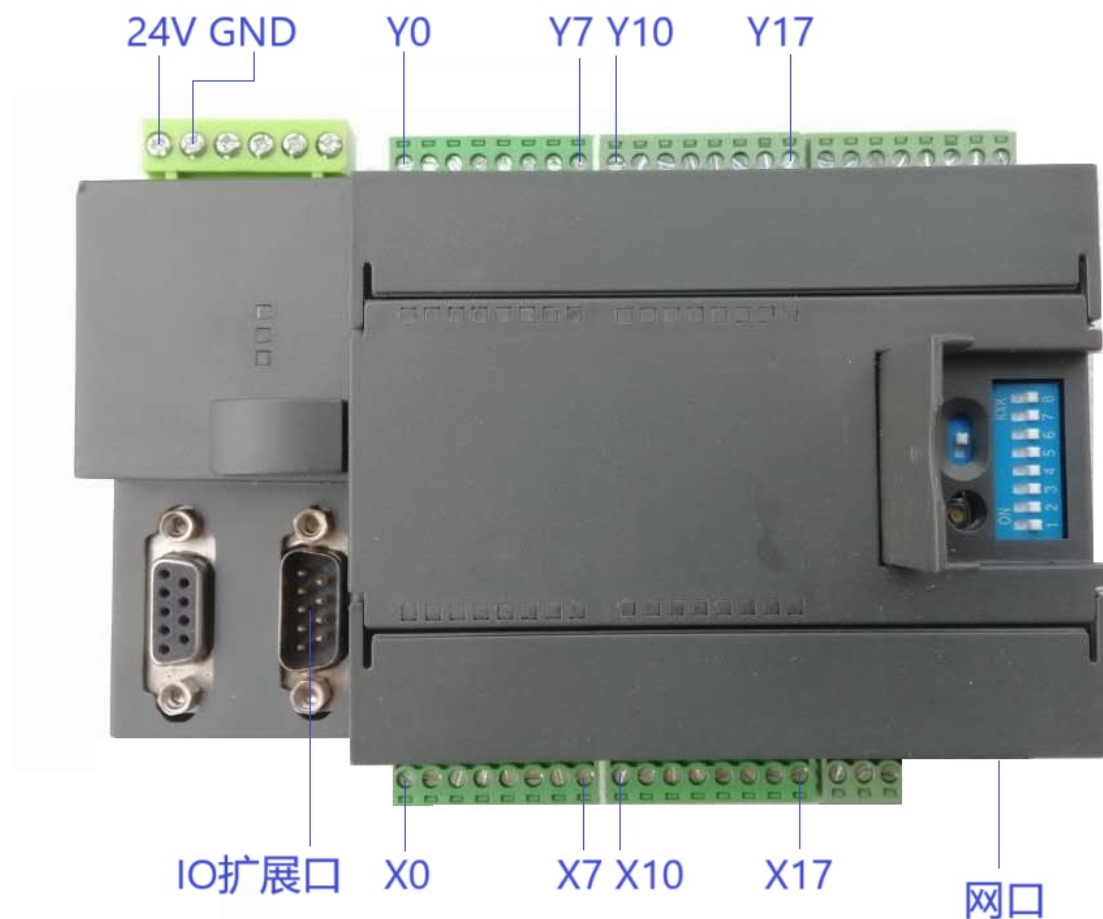


目录

一、硬件资源	3
二、软件资源	4
三、 硬件连接	5
3.1、通用输出 IO 接线方式.....	5
3.2、通用输入 IO 接线方式.....	7
四、API 返回值及其意义	8
五、API 使用说明	9
5.1、板卡打开关闭 API.....	9
5.2、IO 操作 API	10
5.3、AD/DA 操作 API.....	13
5.4、其他指令 API.....	14
六、Demo 软件 DemoVC	15
七、PC 端 IP 配置及多轴板卡并联实现方法	16
八、IO 扩展方法	17
九、运动控制卡安装尺寸	18
十、附录 API 一览.....	19
十一、常见问题解答	20
11.1、如何修改 IP 地址?	20
11.2、IP 地址忘记了怎么办?	20

一、硬件资源

- 1、控制卡自身有 16 路通用输入、16 路通用输出，输入采用光耦隔离，输出可直接驱动继电器。
- 2、控制卡支持 IO 扩展，可以扩展为 32、48、64、128、256、512、1024.....路输入输出。
- 3、控制卡采用 24V 直流电源供电。
- 4、控制卡带有 1 路以太网口。
- 5、控制卡有 2 路 AD,2 路 DA,模拟量范围均为 0~10V。



二、软件资源

控制卡提供了 VC++及 C#下的动态库、头文件，提供例程源代码，用户可利用动态库提供的 API 完成板卡打开、关闭、IO 输入输出。Labview 下也可以使用。同时板卡支持 Linux、Android、iOS、Wince、Python 等开发环境及语言。

名称	修改日期	类型	大小
测试软件-EthBoardTest_2016	2019/1/23 17:44	文件夹	
开发例程源代码 (基于VS2010)	2019/1/23 17:44	文件夹	
库文件及头文件	2019/1/23 17:44	文件夹	
8轴运动控制卡用户手册V2.0.pdf	2018/7/1 14:46	Adobe Acrobat ...	1,663 KB
开发工具软件VS2010下载地址 (百度云盘链接) .txt	2018/7/24 20:55	文本文档	1 KB

三、硬件连接

3.1、通用输出 IO 接线方式

注:千万不可将 24V 直接接入输出 IO，可能导致输出端口短路，烧坏板卡！！！！

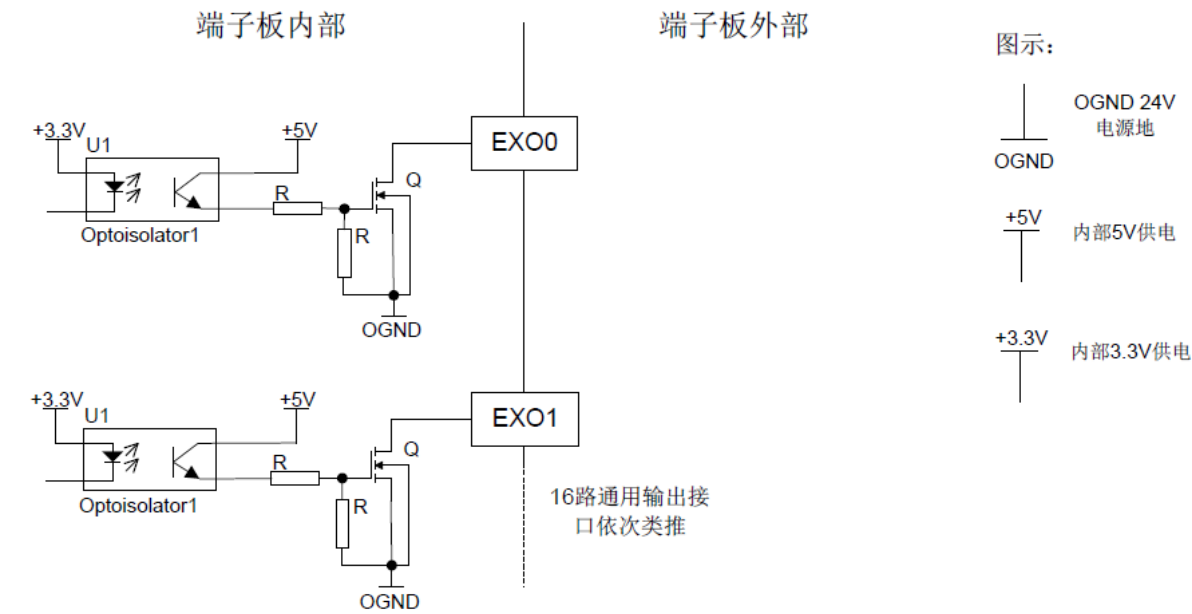
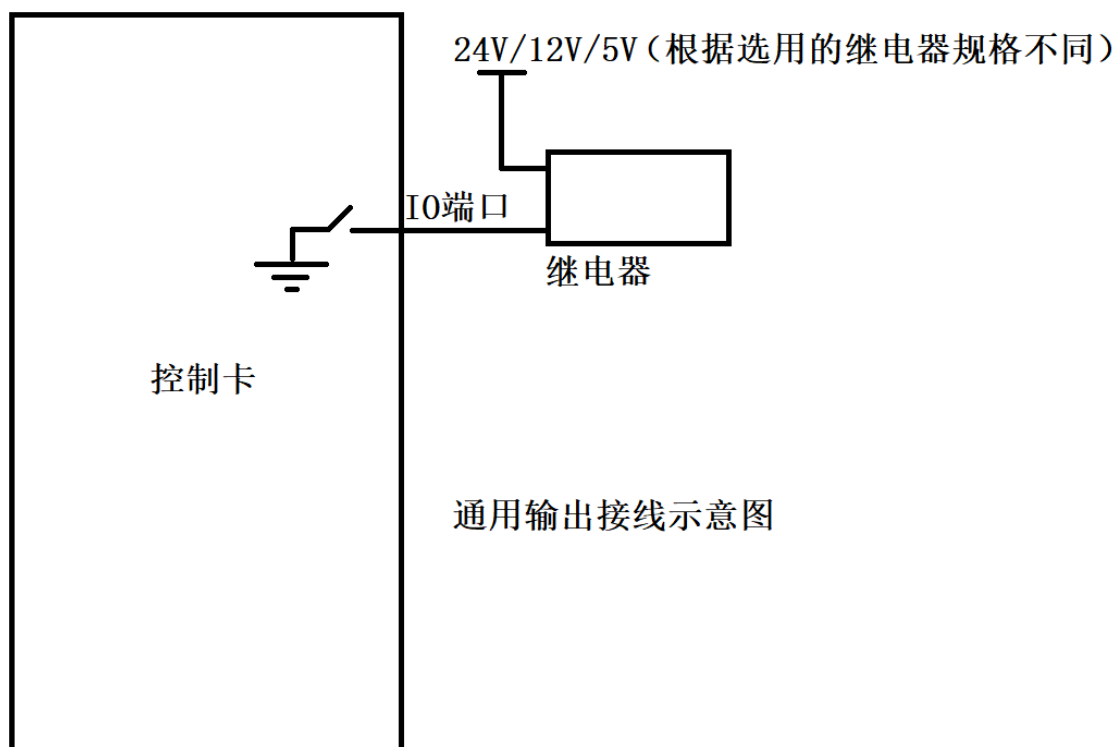


图 3-3：端子板通用数字输出信号内部电路示意图



3.2、通用输入 IO 接线方式

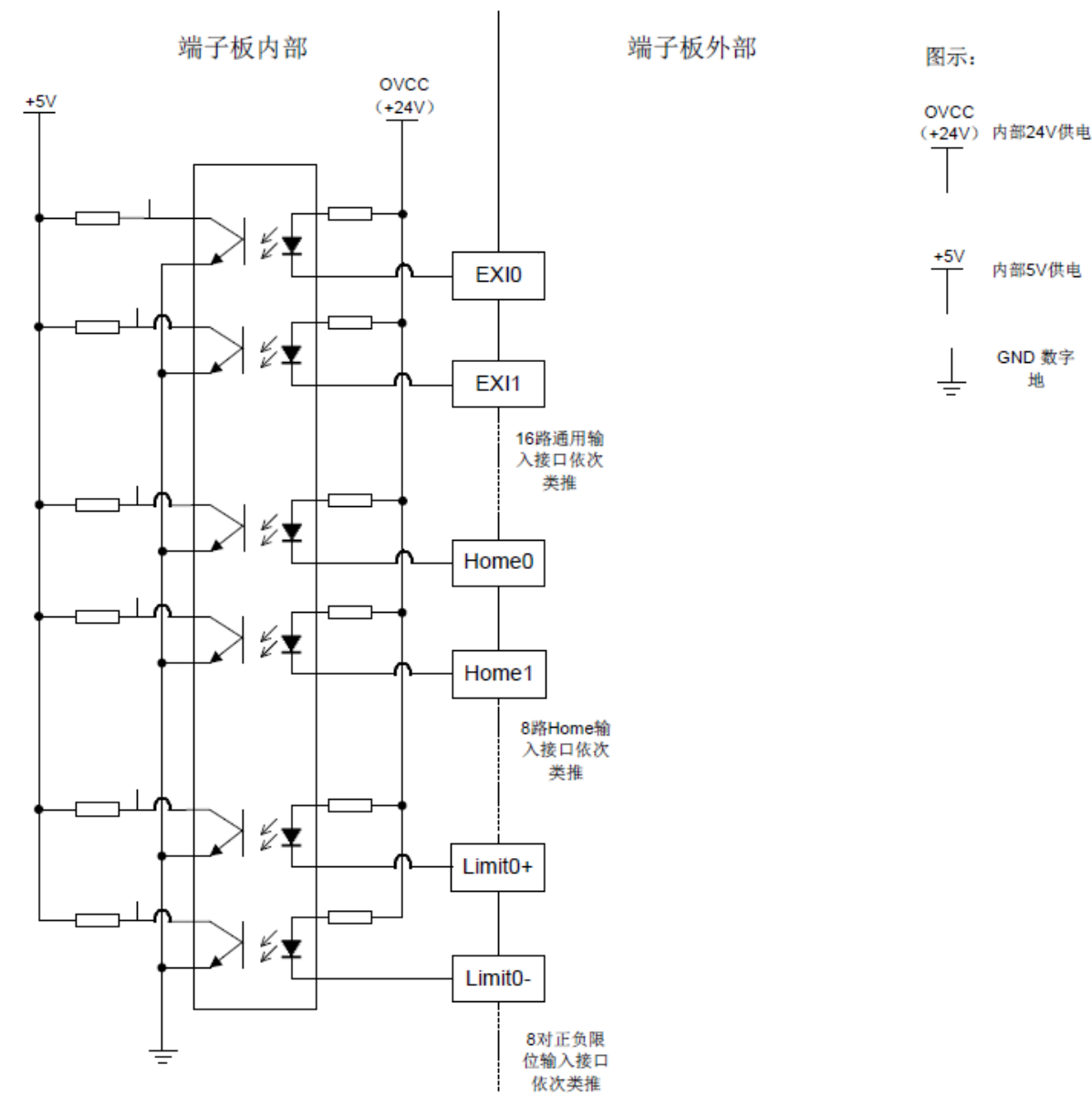
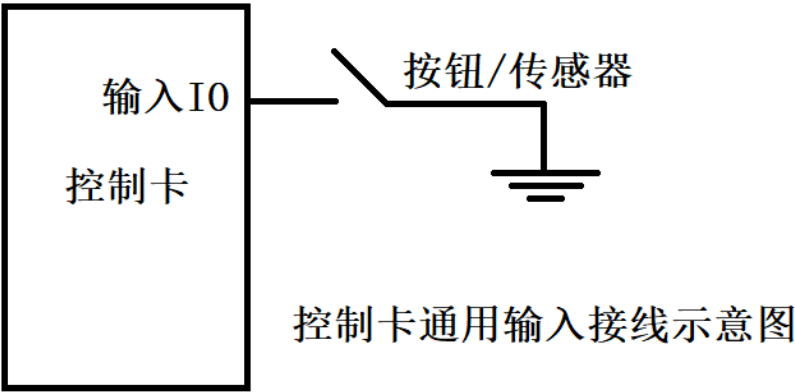


图 3-2：端子板通用输入，输入信号内部电路示意图



四、API 返回值及其意义

返回值	意义	处理方法
0	执行成功	
1	执行失败	检测命令执行条件是否满足
2	版本不支持该 API	如有需要，联系厂家
7	参数错误	检测参数是否合理
-1	通讯失败	接线是否牢靠，更换板卡
-6	打开控制器失败	是否输入正确串口名，是否调用 2 次 MC_Open
-7	运动控制器无响应	检测运动控制器是否连接，是否打开。更换板卡

五、API 使用说明

5.1、板卡打开关闭 API

API	说明
MC_Open	打开板卡 (注意打开板卡函数，不同语言略有不同，以对应语言的例程为准)
MC_Reset	复位板卡
MC_Close	关闭板卡

参数详细说明：

int MC_Open(short iType=0, char* cName="COM1")	
iType	打开方式，0 网口，1 串口
cName	当 iType=0（网口方式打开）时，该参数代表 PC 端 IP 地址 当 iType=1（串口方式打开）时，该参数代表默认串口号
int MC_OpenByIP(char* cPCIP, char* cCardIP, unsigned long ulID, unsigned short nRetryTime)	
cPCIP	电脑 IP
cCardIP	板卡 IP
ulID	固定为 0
nRetryTime	固定为 0
int MC_Open(short nCardNum, char* cPCEthernetIP, unsigned short nPCEthernetPort, char* cCardEthernetIP, unsigned short nCardEthernetPort) (仅网口的 C++/C#语言支持该方式)	
nCardNum	卡号，从 1 开始，第一个卡填 1，第二个卡填 2.....
cPCEthernetIP	电脑 IP 地址（需要和板卡在同一个网段）
nPCEthernetPort	电脑端口号，通常第一个卡填 60000，第二个卡填 60001.....
cCardEthernetIP	板卡 IP 地址（出厂默认 192.168.0.1）
nCardEthernetPort	板卡端口号（和电脑端口号保持一致即可）
int MC_Reset()	
无参数	
int MC_Close()	
无参数	

示例代码：

```
int iRes = 0;
```

```
iRes += MC_Open(0, "192.168.0.200");//打开板卡(通过网口，PC 端 IP 地址为 192.168.0.200)
```

```
iRes += MC_Reset();//复位板卡
```

5.2、IO 操作 API

API	说明
MC_GetDiRaw	获取 IO 输入（包含主卡 IO、限位、零位）
MC_SetExtDoValue	设置 IO 输出（包含主模块和扩展模块）
MC_GetExtDiValue	获取 IO 输入（包含主模块和扩展模块）
MC_GetExtDoValue	获取 IO 输出（包含主模块和扩展模块）
MC_SetExtDoBit	设置指定 IO 模块的指定位输出（包含主模块和扩展模块）
MC_GetExtDiBit	获取指定 IO 模块的指定位输入（包含主模块和扩展模块）
MC_GetExtDoBit	获取指定 IO 模块的指定位输出（包含主模块和扩展模块）
MC_GetDiReverseCount	获取指定模块指定 IO 的变化次数或者上升沿/下降沿次数
MC_SetDiReverseCount	设置指定模块指定 IO 的变化次数
MC_GpioCmpBufData	向 GPIO 比较器缓冲区发送比较数据
MC_GpioCmpBufStop	停止 GPIO 比较缓冲区

参数详细说明：

int MC_GetDiRaw(short diType, long *pValue);	
diType	指定数字 IO 类型 MC_GPI(该宏定义为 4) 通用输入
pValue	IO 输入值存放指针
int MC_SetExtDoValue(short nCardIndex, unsigned long *value, short nCount=1)	
nCardIndex	起始板卡索引(0~63), 0 是主模块, 扩展模块从 1 开始
value	IO 输出值存放指针
nCount	本次设置的模块数量(1~64)
int MC_GetExtDiValue(short nCardIndex, unsigned long *pValue, short nCount=1)	
nCardIndex	起始板卡索引(0~63), 0 是主模块, 扩展模块从 1 开始
pValue	IO 输入值存放指针
nCount	本次获取的模块数量(1~64)
int MC_GetExtDoValue(short nCardIndex, unsigned long *pValue, short nCount=1)	
nCardIndex	起始板卡索引(0~63), 0 是主模块, 扩展模块从 1 开始
pValue	IO 输出值存放指针
nCount	本次获取的模块数量(1~64)
int MC_SetExtDoBit(short nCardIndex, short nBitIndex, unsigned short nValue)	
nCardIndex	起始板卡索引(0~63), 0 是主模块, 扩展模块从 1 开始
nBitIndex	IO 位索引号(0~15)
nValue	IO 输出值(0/1)
int MC_GetExtDiBit(short nCardIndex, short nBitIndex, unsigned short *pValue)	
nCardIndex	起始板卡索引(0~63), 0 是主模块, 扩展模块从 1 开始
nBitIndex	IO 位索引号(0~15)
pValue	IO 输入值存放指针
int MC_GetExtDoBit(short nCardIndex, short nBitIndex, unsigned short *pValue)	
nCardIndex	起始板卡索引(0~63), 0 是主模块, 扩展模块从 1 开始
nBitIndex	IO 位索引号(0~15)
pValue	IO 输入值存放指针

int MC_GetDiReverseCount(short nDiType, short diIndex, int * pReverseCount, short nCount, short nCardIndex, short nRiseSinkType)	
nDiType	固定为 4
diIndex	I0 起始位索引号 (0~15)
pReverseCount	I0 变化次数存放指针
nCount	获取的 I0 数量
nCardIndex	卡号, 主卡为 0, 扩展卡为 1、2、3.....
nRiseSinkType	0 获取上升沿和下降沿, 1 获取上升沿, -1 获取下降沿
int MC_SetDiReverseCount(short nDiType, short diIndex, int ReverseCount, short nCount, short nCardIndex)	
nDiType	固定为 4
diIndex	I0 起始位索引号 (0~15)
ReverseCount	要设置的初始值, 通常可以初始化为 0
nCount	要设置的 I0 数量
nCardIndex	卡号, 主卡为 0, 扩展卡为 1、2、3.....
int MC_GpioCmpBufData(short nCmpGpoIndex, short nCmpSource, short nPluseType, short nStartLevel, short nTime, short nTimerFlag, short nAbsPosFlag, short nBufLen, long *pBuf)	
nCmpGpoIndex	输出 I0 索引, 0~47 0 代表 Y0 1 代表 Y1 2 代表 Y2 以此类推.....
nCmpSource	比较数据来源 1000:X0 下降沿次数 1001:X1 下降沿次数 1002:X2 下降沿次数 1003:X3 下降沿次数 以此类推..... 1100:X0 上升沿次数 1101:X1 上升沿次数 1102:X2 上升沿次数 1103:X3 上升沿次数 以此类推.....
nPluseType	预留, 固定为 2
nStartLevel	预留, 固定为 0
nTime	输出 I0 脉冲时间, 单位 ms
nTimerFlag	固定为 1
nAbsPosFlag	0 相对, 1 绝对
nBufLen	待比较数据长度, 1~32, 通常为 1
pBuf	待比较数据存放指针
int MC_GpioCmpBufStop(long *pGpioMask, short nCount)	
pGpioMask	要停止比较的 I0 掩码

nCount	模块数量
--------	------

示例代码:

```
int iRes = 0;
```

```
iRes += MC_Open(0, "192.168.0.200");//打开板卡（通过网口，PC端IP地址为192.168.0.200）
```

```
iRes += MC_Reset();//复位板卡
```

```
iRes += MC_SetExtDoBit(0, 4, 1);//设置板卡1端口3IO输出
```

```
iRes += MC_SetExtDoBit(0, 4, 0);//关闭板卡1端口3IO输出
```

5.3、AD/DA 操作 API

API	说明
MC_SetDac	设置 dac 输出电压
MC_GetAdc	读取 nAdcNum 输出电压
MC_SetExDac	设置 dac 输出电压（包括主卡和扩展卡）
MC_GetExAdc	读取 nAdcNum 输出电压（包括主卡和扩展卡）

参数详细说明：

int MC_SetDac(short nDacNum, short* pValue, short nCount=1)	
nDacNum	dac 起始通道号
Value	输出电压 0 对应 0V；10000 对应+10V
nCount	设置的通道数，默认为 1，1 次最多可以设置 8 路 DAC
int MC_GetAdc(short nAdcNum, short *pValue, short nCount=1, unsigned long *pClock=NULL)	
nAdcNum	ADC 通道号
pValue	ADC 通道电压, 0 对应 0V；10000 对应+10V
nCount	读取的通道数，默认为 1，1 次最多可以读取 8 个 ADC
int MC_SetExDac(short nCardIndex, short nDacNum, short* pValue, short nCount=1)	
nCardIndex	0: 主卡, 1 扩展卡 1.....
nDacNum	dac 起始通道号
Value	输出电压 0 对应 0V；10000 对应+10V
nCount	设置的通道数，默认为 1，1 次最多可以设置 8 路 DAC
int MC_GetExAdc(short nCardIndex, short nADCNum, short *pValue, short nCount=1, unsigned long *pClock=NULL)	
nCardIndex	0: 主卡, 1 扩展卡 1.....
nAdcNum	ADC 通道号
pValue	ADC 通道电压, 0 对应 0V；10000 对应+10V
nCount	读取的通道数，默认为 1，1 次最多可以读取 2 个 ADC

示例代码：

```
int iRes = 0;
short nValue = 0;

iRes += MC_Open(0, "192.168.0.200");//打开板卡（通过网口，PC 端 IP 地址为 192.168.0.200）
iRes += MC_Reset();//复位板卡

nValue = 5000;
//DAC1 输出 5V
iRes += MC_SetDac(1, &nValue, 1);
iRes += MC_GetAdc(1, &nValue, 1);
```

5.4、其他指令 API

API	说明
MC_GetID	获取板卡唯一标识 ID
MC_SetKeepAlive	设置通讯心跳时间

参数详细说明:

int MC_GetID(unsigned long* pID)	
pID	指向存放 ID 地址的指针
int MC_SetKeepAlive(long AliveTime, long ulEnableMask, long ulEnableValue, short* nAdcMask, short* nAdcValue, long* ulIOMask, long* ulIOValue, short nModuleNum)	
long AliveTime	通讯最大间隔时间，超过该时间没有收到上位机数据包，自动停止所有轴运动并且关闭指定 IO 和模拟量 单位毫秒
long ulEnableMask	超时要操作使能的轴掩码，Bit0 代表轴 1，Bit1 代表轴 2.....
long ulEnableValue	超时要操作使能的轴值，Bit0 代表轴 1，Bit1 代表轴 2.....
short* nAdcMask	超时要操作的模拟量掩码，Bit0 代表模拟量 1，Bit1 代表模拟量 2.....
short* nAdcValue	超时要操作的模拟量值
long* ulIOMask	超时要操作的 IO 掩码，Bit0 代表 IO1，Bit1 代表轴 IO2.....
long* ulIOValue	超时要操作的 IO 值，Bit0 代表 IO1，Bit1 代表轴 IO2.....
short nModuleNum	总模块数量（包含主模块）

示例代码:

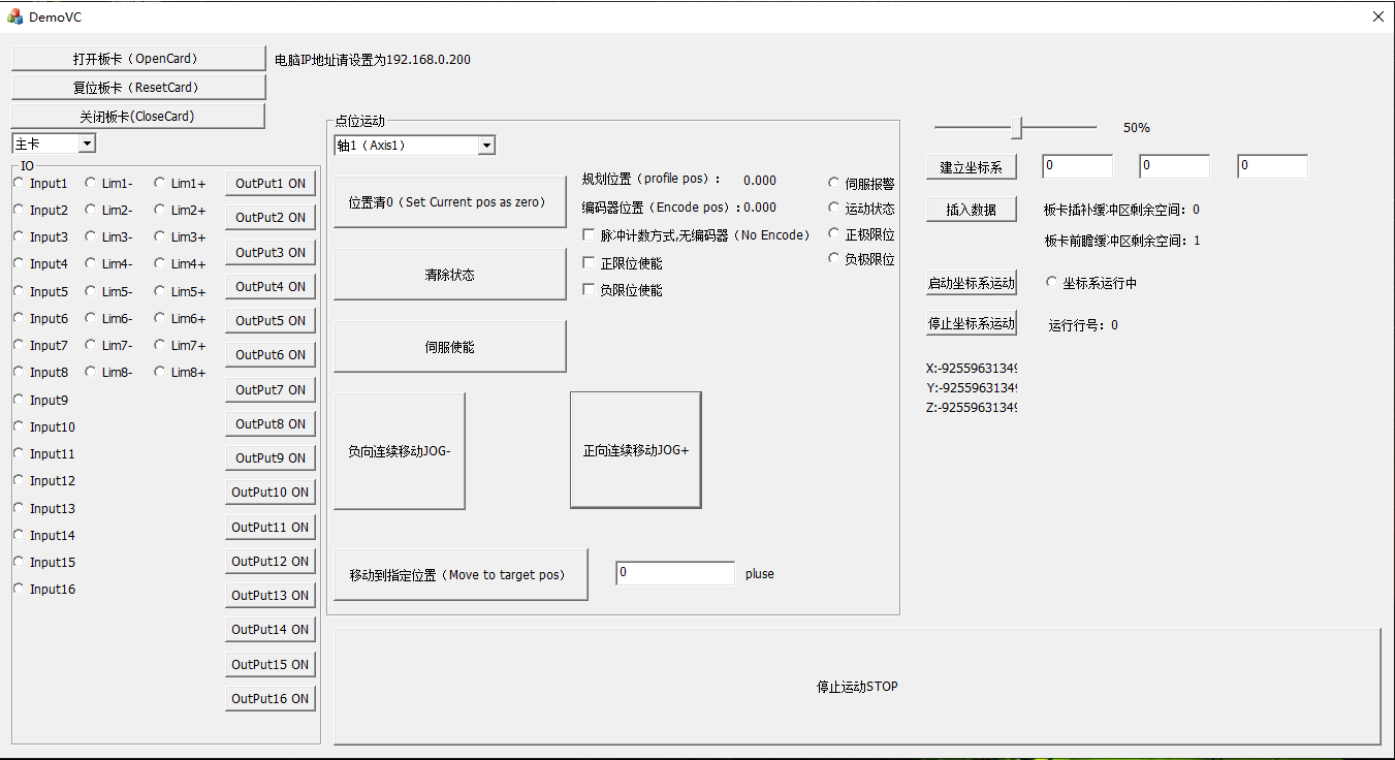
```
int iRes = 0;
long lSts = 0;
unsigned long ulID = 0;

long lSoftLimPos= 0X7FFFFFFF;
long lSoftLimNeg=0X80000000

iRes += MC_Open(0, "192.168.0.200");//打开板卡（通过网口，PC 端 IP 地址为 192.168.0.200）
iRes += MC_Reset();//复位板卡

iRes = MC_GetID(&ulID);
```

六、Demo 软件 DemoVC



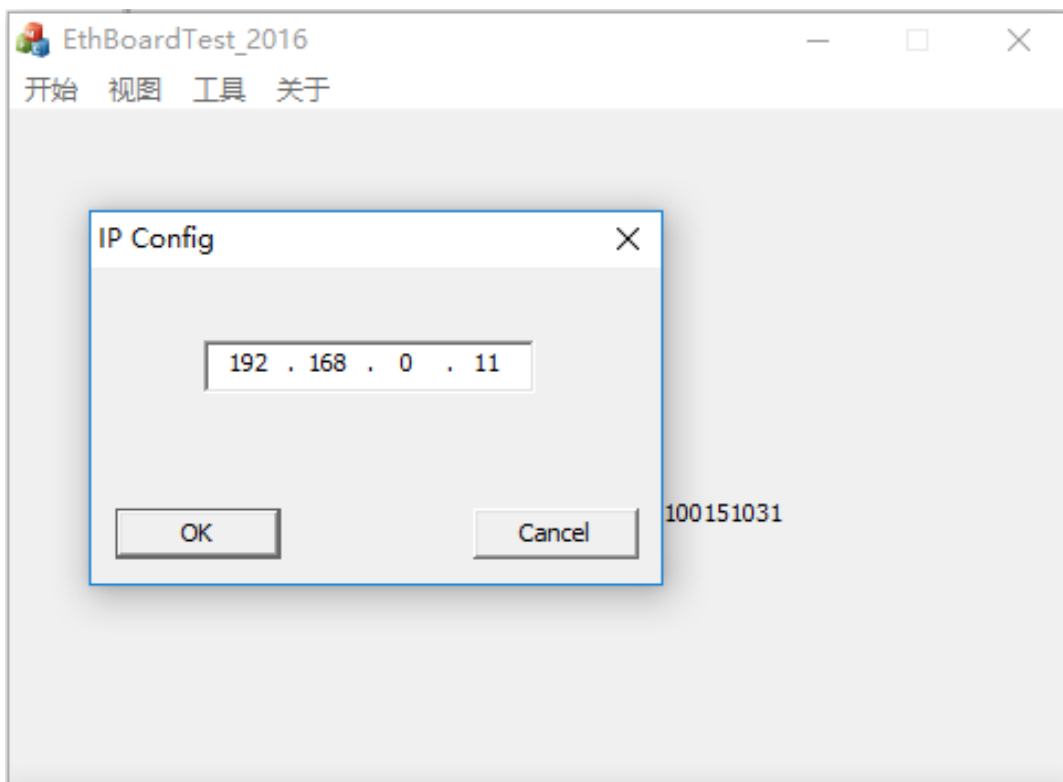
七、PC 端 IP 配置及多轴板卡并联实现方法

通过多个板卡并联的方案，一台电脑可以控制 32、64、128、1024 个 IO
电脑 IP 地址需设置为 192.168.0.200

板卡出厂默认 IP 地址为 192.168.0.1，通常情况下无需做修改。

打开板卡代码为：`MC_Open(0, "192.168.0.200")`；

用户如果需要多个轴板卡同时使用，可以用串口方式打开板卡，修改 IP 地址。如下图所示：



重点说明：如果使用交换机，通常建议板卡 IP 从 192.168.0.2 开始。

八、IO 扩展方法

如果运动控制卡自身 IO 不够用，可选购我司 IO 扩展卡配套使用，通过一根 DB9 延长线串联即可，扩展协议为我司自定义总线协议，性能稳定，延迟小，且不占用电脑端口。

九、运动控制卡安装尺寸

卡规安装，外形尺寸 141mm*80mm*62mm

十、附录 API 一览

板卡打开关闭 API	
MC_Open	打开板卡
MC_Reset	复位板卡
MC_Close	关闭板卡
IO 操作 API	
MC_SetExtDoValue	设置 IO 输出（包含主模块和扩展模块）
MC_GetExtDiValue	获取 IO 输入（包含主模块和扩展模块）
MC_GetExtDoValue	获取 IO 输出（包含主模块和扩展模块）
MC_SetExtDoBit	设置指定 IO 模块的指定位输出（包含主模块和扩展模块）
MC_GetExtDiBit	获取指定 IO 模块的指定位输入（包含主模块和扩展模块）
MC_GetExtDoBit	获取指定 IO 模块的指定位输出（包含主模块和扩展模块）
MC_GetDiReverseCount	获取指定模块指定 IO 的变化次数或者上升沿/下降沿次数
MC_SetDiReverseCount	设置指定模块指定 IO 的变化次数初始值
其他 API	
MC_GetID	获取板卡唯一标识 ID
DA/AD 类 API	
MC_SetDac	设置 dac 输出电压
MC_GetAdc	读取 nAdcNum 输出电压

十一、常见问题解答

11.1、如何修改 IP 地址？

链接: <https://pan.baidu.com/s/1f1gupFRGjVr8E4dLOPrsZg>

提取码: pr6c

下载上面修改 IP 地址的小工具即可修改 IP 地址。

如果 IP 改为 192.168.0.XXX, 以上操作完成后即可。

如果 IP 不是 192.168.0.XXX, 比如想修改为 129.186.1.3, 则还需要完成以下步骤:

- 1、确认电脑 IP 地址, 例如电脑 IP 地址为 129.186.1.200
- 2、设置板卡 IP 地址为电脑网段如: 129.186.1.3
- 3、修改 CardIPConfig.txt 里面的 CardIP_1=129.186.1.3

完成以上步骤后重启板卡即可, 打开板卡代码为 MC_Open(0,"129.186.1.200")

备注:

- 1、如果忘记了板卡 IP 地址, 长按复位按键可恢复出厂设置 192.168.0.1。
- 2、CardIPConfig.txt 这个文件可在 IP 修改工具里面找到, 需 COPY 到自己的应用程序所在目录。

11.2、IP 地址忘记了怎么办？

上电状态下, 长按面板上的复位按键就可以恢复出厂 IP, 出厂 IP 地址为 192.168.0.1。